Greedy and Friends!

Workshop – Term 1, Week 3

Susan He, Austin Song

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 Problem examples

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food

VJudge For Workshop

If you're confident with the content for today, feel free to get started on problems!

Link: https://vjudge.net/contest/793877

Password: `cpmsoc`

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 Problem examples

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

A Problem

Making Change

You have infinite \$20, \$5, and \$1 coins. How can you make some positive integer amount X with as few coins as possible?

. . . And Its Solution

- Best thing to do is to play through the problem!
- Let's try and make \$37.
  - First take a \$20 coin?
  - Have to make \$17 now; let's use three \$5 and two \$1s.
- Used 6 coins... is this the best we can do?

General strategy:

- Take as many \$20 coins as you can ($\lfloor X/20 \rfloor$).
- Take as many \$5 coins after that ($\lfloor (X \bmod 20)/5 \rfloor$).
- Whatever's left must be \$1 coins ($X \bmod 5$).

This is a **greedy** strategy.

The "Greedy" Paradigm

Definition (Greedy Agorithms)

In a greedy algorithm, we always take the **locally optimal** solution.

A greedy algorithm looks like:

1. Quickly computing the best move at the moment (local optima),
   - e.g. Finding the greatest coin value we can still take without overshooting X.
2. Rashly performing this move.

Note that this isn't a single algorithm; moreso a general philosophy.

Benefits of a standard greedy:

- Very fast – we don't need to compute the future; just look at current position.
- Easy to think about – often the intuitive approach to a problem.

So are there any downsides?

Does This Always Work?

Making Change II

You have infinite \$20, \$6, and \$1 coins. How can you make some positive integer amount X with as few coins as possible?

Let's try and make \$24.

- We take as many \$20 as possible (i.e. one \$20 coin).
- We have \$4 left, so we can use four \$1 coins.
- This means we only need to use 5 coins! *. . . is what someone incorrect would say.*
- You can use 4 \$6 coins to make \$24 instead.

Since we can't regret our choices (else we lose our speed benefit!), greedy is **shortsighted**.

If you're more advanced, you can try solve this problem algorithmically! (hint: look for $O(N)$)

The Gamble

Greedy doesn't always work then – so when can we use greedy?

We can use it if we can prove being shortsighted *always* results in a global optima!

The most common way we do this is through an **exchange argument**.

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 **Proofs via exchange arguments**

4 Problem examples

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

Social Distancing

Problem

You're preparing a very unique banquet. There's a long table with L seats (numbered from 1 to L), and N meals on this table – the i'th meal is located at the a_i'th seat. When people come in, you choose a dish for them to sit at (one dish per person). However, there's a crucial rule:

"No two people can be less than K seats apart."

What's the maximum number of people you can invite so that everyone gets a dish?

Constraints:

- $1 \leq N \leq 10^6$, $1 \leq K \leq L \leq 10^9$, $1 \leq a_i \leq L$.

Social Distancing – Samples

Sample Cases

Sample Input:

```
5 4
1 3 5 10 12
```

Sample Output:

```
3
```

A Bold Assertion!

We should always assign the first person to the first dish. <o>_<o>

Proof: Suppose a better solution involves **not taking** the first dish:

- This means the second person must be assigned to some seat $\geq a_2$.
- There are two cases to consider:
  1. Suppose $a_2 - a_1 \geq K$.
     - Then, we should be allowed to seat someone at a_1 anyway.
     - This increases the number of people we can invite by 1.
     - Thus, not taking a_1 can't have been better.
  2. Suppose $a_2 - a_1 < K$.
     - If we placed the guy at a seat $\geq a_2$, the next guy has to be in a seat $\geq a_2 + K$.
     - Instead, if we place the guy at a_1, the next guy only needs to be in a seat $\geq a_1 + K$.
     - Since $a_1 + K \leq a_2 + K$, by moving the first person to the first dish, we don't lose a person, *and* we leave more space for other people!
- We reached a contradiction in both cases – there's no better solution that doesn't take the first dish!

So What Happened?

Our proof came in a few major components:

1. We came up with a greedy **invariant** – something that's always true.
   - e.g. We should always take the first dish.
2. We assumed (by way of contradiction) that there's some better solution that doesn't satisfy the condition.
3. We **exchanged** a part of this solution so that it sticks to the invariant (i.e. exchanged one of its choices for a greedy choice).
   - e.g. We either added in a person at a_1, or we shifted the first person to a_1.
4. We showed that by making this exchange, our score never gets worse.
   - Since there's no *better* solution, the greedy must be the best!

Exercise: Using this, try find a full greedy algorithm for the problem!

(**Hint**: Try prove that you should always take the first *available* dish at any given time.)

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 **Problem examples**

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

Making Change – Proof

Our Assumption

You can get an optimal solution by choosing the maximum number of coins of each denomination (descending).

Let's assume there's some better solution which doesn't do this.

- Let's say for simplification sake it doesn't use an extra \$20 while the greedy does.
- Imagine there are x \$5 coins and y \$1 coins.
- Since the money we need to make *must* exceed \$20 (else both greedy and this solution would not have taken any \$20s), we have that $5x + y \geq 20$.
- Convince yourself that you can pick some of these coins and convert them into \$20s.
- The same argument works for \$5 (try it!)
- Contradiction! The assumed 'better' solution can't ever be better. The greedy works!

Making Change II – Proof

Why does the same proof not work for this problem?

(Reminder: coin denominations of 20, 6, 1)

Gaming

Problem

There are N games you want to play. The i'th game can only be started on day s_i, and takes t_i days to finish. You're a completionist, and will only play **one** game at a time. Furthermore, once you've started a game, you cannot stop until it's done! Note that you cannot start a game at the same time another game finishes.

What's the maximum number of games you can play?

Constraints:

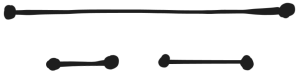
- $1 \leq N \leq 10^5$, $1 \leq s_i \leq t_i \leq 10^9$.

Gaming – Discussion

We can think of each game as an interval $[s_i, s_i + t_i]$.

The problem then becomes: what's the largest set of non intersecting intervals we can pick?

Maybe pick the earliest start time always?



Maybe pick the intervals with the shortest duration at every point?

Maybe pick the intervals with the least intersections?

Turns out, none of these work!

Gaming – Solution

Let's hypothesise that we should instead pick the interval with **earliest finish time**.

Assume there exists a better solution where this doesn't hold.

- Call the interval with earliest finish time x, and the set of intervals that intersect it Y.
- Assume that $|Y| > 0$ and the algorithm picks some $y \in Y$. (what if it's not true?)
- Since x has the earliest finish time, all $y \in Y$ must start $\leq s_x + t_x$ and end $\geq s_x + t_x$.
- But this means they all intersect each other – can only choose *one* interval in $\{x\} \cup Y$.
- By performing an exchange from some $y \in Y$ picked by this solution, to x, we don't ever decrease the score! The solution can't be better (it's the same).
- Since there's no better solution, this condition must be true!

Exercise: Show (inductively or otherwise) that you should do this at every step.

Gaming – Implementation?

How do we implement?

1. We sort all intervals by their $s_i + t_i$.
2. Let's store the current time $t := \tau = 1$.
3. We start going through intervals:
   - If any interval has $s_i < \tau$, we are already too late, and should skip it.
   - The first interval for which $s_i \geq \tau$ should be the first *allowed* interval with lowest $s_i + t_i$.
   - We should take this interval and fast-forward our time to $\tau = s_i + t_i + 1$.
   - Repeat until all intervals are exhausted.

This version of the greedy algorithm runs in $O(N \log N)$, fast enough to pass. (Why?)

There's only one issue:

How are we sorting a bunch of intervals by endpoints quickly???

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 Problem examples

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

What Is Sorting?

Say that you have an array of numbers A. The numbers aren't necessarily in order. How do we rearrange A so that they're in ascending order?

One common way is via **swaps**. Consider two elements of the array a_i and a_j where $i < j$:

- If $a_i \leq a_j$, we keep the order.
- If $a_i > a_j$, the elements are out of order; we should swap!

Sorting algorithms which compare two elements at every step are called **comparison-based**.

Comparison-Based Sorts

There are various comparison-based sorting algorithms with various time complexities. Notably, quicksort/mergesort are $O(N \log N)$ where N is the size of A. Can we do better?

Turns out – no! In a sort, we need to find the correct permutation out of $N!$ total permutations:

$$\text{“Is } a_i \leq a_j\text{?”}$$

How many cases does one of these questions eliminate?

When we fix a pair's order, half as many permutations remain! Total number of swaps is

$$\log N! = \log(N(N-1)\cdots 1) \geq \log \overbrace{\left(\frac{N}{2} \cdots \frac{N}{2}\right)}^{\frac{N}{2}} = \frac{N}{2} \log\left(\frac{N}{2}\right) \sim O(N \log N)$$

So no matter what, our best (classical) comparison-based algorithms must be $O(N \log N)$.

Why Care?

Since we already know of $O(N \log N)$ algorithms, we can simply blackbox the sort!

- In Python, either `sorted()` or `list.sort()` work.
- In C++, `sort(arr.begin(), arr.end())` works.

As competitive programmers, we usually don't ever need to care about the specific sorting algorithm, as long as it's $O(N \log N)$.

Custom Comparators!

Most times, we are sorting numbers by value. But what if we need to sort more complicated objects?

- e.g. Sorting intervals (s_i, f_i) by ascending f_i.

To do this programmatically, we can write custom comparators!

Passing Function

```
#include<bits/stdc++.h>
using namespace std;

bool compare(pair<int, int> a, pair<int, int> b) {
    return a.second < b.second;
}

int main() {
    vector<pair<int, int>> v = {{1, 3}, {2, 2}, {3, 1}, {3, 3}};
    sort(v.begin(), v.end(), compare);
}
```

Overloading Operator

```
#include<bits/stdc++.h>
using namespace std;

struct Interval {
    int s, f;
    bool operator<(const Interval& other) const {
        return s < other.s;
    }
};

int main() {
    vector<Interval> v = {{1, 3}, {2, 2}, {3, 1}, {3, 3}};
    sort(v.begin(), v.end());
}
```

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 Problem examples

5 Interlude on sorting

6 **Implementation demonstration**

7 Appendix and resources

Gaming – Implementation

Time to implement :D

Outline

1 Housekeeping

2 Introduction to algorithmic hedonism

3 Proofs via exchange arguments

4 Problem examples

5 Interlude on sorting

6 Implementation demonstration

7 Appendix and resources

Upcoming Events

Look out for our **IMC Contest** April 12th!

If you want to get into competitive programming, or prepare for ICPC, find a team and sign up for **SPAR 1** on 14th March!

If you participated in the O-Week Horse contest, **prizes + debrief** tomorrow!

Interested in some maths? Head to our **competitive maths** workshop this Friday!

Feedback!

Please fill out feedback form so we know how to run future workshops :D

(a) Feedback form

(b) Attendance form

Arc CLUBS SUPPORTED BY Arc INDEPENDENTLY RUN

UNSW CPMSoc